

Теоретический материал по теме: List Comprehensions

1. Что такое List Comprehension?

List Comprehension — это компактный способ создания и преобразования списков в Python. Он позволяет выполнять операции над элементами и добавлять их в новый список, используя краткий и удобный синтаксис.

Преимущества List Comprehensions:

- **Упрощает код:** делает его более читаемым и компактным.
- **Быстрее традиционного цикла:** в большинстве случаев List Comprehensions работают быстрее, чем циклы `for` для создания новых списков.

Синтаксис:

```
[выражение for элемент in итерируемый_объект]
```

2. Основы использования List Comprehensions

Пример: Создание списка квадратов чисел

Задача: создать список квадратов чисел от 1 до 5.

```
squares = [x ** 2 for x in range(1, 6)]  
print(squares) # [1, 4, 9, 16, 25]
```

Как работает List Comprehension:

- `for x in range(1, 6)` — перебирает числа от 1 до 5.
- `x ** 2` — выражение, которое возводит каждое число `x` в квадрат и добавляет результат в новый список.

3. Добавление условий (if) в List Comprehensions

List Comprehensions позволяют добавлять условия для фильтрации элементов, используя `if`.

Пример: Список чётных чисел

Задача: создать список чётных чисел от 1 до 10.

```
evens = [x for x in range(1, 11) if x % 2 == 0]  
print(evens) # [2, 4, 6, 8, 10]
```

Объяснение: `if x % 2 == 0` проверяет, является ли число чётным, и добавляет только чётные числа в новый список.

4. Использование if-else внутри List Comprehensions

`if-else` в `List Comprehensions` позволяет добавлять альтернативные значения для элементов, которые не соответствуют условию.

Пример: Чётные и нечётные числа

Задача: создать список, в котором чётные числа заменяются на строку "чётное", а нечётные остаются неизменными.

```
result = ["чётное" if x % 2 == 0 else x for x in range(1, 6)]
print(result) # [1, "чётное", 3, "чётное", 5]
```

Объяснение: `if-else` используется перед `for` и применяется к каждому элементу, создавая новый список с модифицированными значениями.

5. Вложенные List Comprehensions

Вложенные `List Comprehensions` позволяют создавать многомерные списки и обрабатывать сложные структуры данных, такие как матрицы.

Пример: Таблица умножения

Задача: создать таблицу умножения от 1 до 3 в виде списка списков.

```
table = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print(table) # [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

Объяснение: Внешний цикл `for i in range(1, 4)` создаёт строки таблицы, а внутренний цикл `for j in range(1, 4)` создаёт значения в каждой строке.

Примеры практического применения List Comprehensions

Фильтрация списка: Создайте список, содержащий только положительные числа из исходного списка.

```
numbers = [-3, 5, -1, 7, -2]
positives = [x for x in numbers if x > 0]
print(positives) # [5, 7]
```

Перевод текста в верхний регистр: Преобразуйте список слов в новый список, где все слова записаны заглавными буквами.

```
words = ["hello", "world"]
uppercase_words = [word.upper() for word in words]
print(uppercase_words) # ["HELLO", "WORLD"]
```

Создание матрицы: Создайте список, представляющий матрицу 3x3, где каждый элемент — это кортеж с индексами строки и столбца.

```
matrix = [(i, j) for j in range(3) for i in range(3)]
print(matrix)
# [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

Основные моменты для запоминания

1. **List Comprehensions** — это способ компактного и удобного создания новых списков на основе существующих итерируемых объектов.
2. **Условия (if)** внутри List Comprehensions позволяют фильтровать элементы, а **if-else** добавляет дополнительные возможности обработки данных.
3. **Вложенные List Comprehensions** помогают работать с многомерными структурами данных, например, для создания таблиц и матриц.